MultiMatch: Multihead Consistency Regularization Matching for Semi-Supervised Text Classification

Iustin Sirbu[1,4], **Robert-Adrian Popovici**[1],
Cornelia Caragea[2], **Stefan Trausan-Matu**[1], **Traian Rebedea**[1,3]

[1]National University of Science and Technology POLITEHNICA Bucharest,
[2]University of Illinois Chicago, [3]NVIDIA, [4]Renius Technologies

iustin.sirbu@upb.ro

Abstract

We introduce **MultiMatch**, a novel semi-supervised learning (SSL) algorithm combining the paradigms of co-training and consistency regularization with pseudo-labeling. At its core, MultiMatch features a pseudo-label weighting module designed for selecting and filtering pseudo-labels based on head agreement and model confidence, and weighting them according to the perceived classification difficulty. This novel module enhances and unifies three existing techniques – heads agreement from **Multi**head Co-training, self-adaptive thresholds from Free**Match**, and Average Pseudo-Margins from Margin**Match** – resulting in a holistic approach that improves robustness and performance in SSL settings. Experimental results on benchmark datasets highlight the superior performance of MultiMatch, i.e., MultiMatch achieves state-of-the-art results on 8 out of 10 setups from 5 natural language processing datasets and ranks first according to the Friedman test among 21 methods. Furthermore, MultiMatch demonstrates exceptional robustness in highly imbalanced settings, outperforming the second-best approach by 3.26%, a critical advantage for real-world text classification tasks. Our code is available on GitHub.

1 Introduction

Deep learning models typically require vast amounts of labeled data to achieve high performance across various tasks (Vaswani, 2017). Moreover, the process of data annotation is both costly and time-intensive, often posing a significant bottleneck in real-world applications. To mitigate this challenge, semi-supervised learning methods (SSL) have emerged as a promising alternative. By leveraging a small set of labeled examples alongside a much larger unlabeled corpus, these approaches greatly reduce the time and resources needed for manual annotation while still maintaining competitive model performance (Berthelot et al., 2019b).

In recent years, semi-supervised learning has been dominated by two main paradigms. The first, pseudo-labeling with consistency regularization (Sohn et al., 2020; Zhang et al., 2022), enforces prediction invariance across different augmentations of the same sample, leveraging high-confidence predictions on weakly augmented data to generate pseudo-labels for strongly augmented counterparts. The second, co-training (Chen et al., 2021; Zou and Caragea, 2023), mitigates error accumulation by training two networks in parallel, each providing pseudo-labels for the other.

Both paradigms have seen significant advancements. In pseudo-labeling with consistency regularization, research has primarily focused on improving pseudo-label filtering, with FreeMatch (Wang et al., 2023) introducing class-wise self-adaptive thresholding and MarginMatch (Sosea and Caragea, 2023) incorporating predictions from past epochs to enhance reliability. In the co-training paradigm, Multihead Co-training (Chen et al., 2021) reduced the need for multiple networks by employing a multi-headed architecture, while JointMatch (Zou and Caragea, 2023) highlighted the importance of samples where models produce different predictions. However, limited research has explored the integration of these paradigms to fully leverage their complementary strengths.

To bridge this gap, we introduce **MultiMatch**, a novel algorithm that incorporates a three-fold pseudo-label weighting module (PLWM). This module integrates the voting mechanism from **Multi**head Co-training (Chen et al., 2021) with a self-adaptive class-wise filtering strategy that leverages both historical and current predictions. Specifically, it incorporates the Average Pseudo-Margin (APM) introduced by Margin**Match** (Sosea and Caragea, 2023) to refine pseudo-label filtering based on historical predictions, while also utilizing the self-adaptive filtering mechanism of Free**Match** (Wang et al., 2023), which dynamically

Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 2792–2808
November 4-9, 2025 ©2025 Association for Computational Linguistics

adjusts thresholds based on the model's current predictions. Additionally, we define a taxonomy of "not useful", "useful & difficult", and "useful & easy" examples, and propose a weighting mechanism based on it.

We evaluate MultiMatch against 20 baselines across five text classification datasets from the unified SSL benchmark – USB (Wang et al., 2022). To replicate the class imbalance challenges commonly encountered in real-world text classification tasks, we also construct highly imbalanced versions of the five datasets. Our algorithm consistently outperforms all baselines, ranking first according to both the Friedman test and the mean error across all datasets, in both balanced and imbalanced settings.

To sum up, our main contributions are: 1) We introduce MultiMatch, a novel SSL algorithm that enhances and combines the strengths of different SSL paradigms; 2) We obtain state-of-the-art performance on five (out of five) datasets widely used for text classification; 3) We show the robustness of MultiMatch on skewed class distributions by obtaining state-of-the-art results in highly imbalanced settings; and 4) We provide a comprehensive ablation study and analysis of our approach.

2 Related Work

Semi-supervised Learning. Following the introduction of FixMatch (Sohn et al., 2020), a widely adopted paradigm in semi-supervised learning has been the combination of consistency regularization and pseudo-labeling. Consistency regularization (Sajjadi et al., 2016) is based on the principle that the predictions of a model should remain invariant under realistic perturbations applied to input samples. Pseudo-labeling (McLachlan, 1975) leverages the model's high-confidence predictions on unlabeled data, treating them as pseudo-labels that are incorporated into the training process.

Early approaches such as FixMatch (Sohn et al., 2020) and UDA (Xie et al., 2020a) utilize a predefined threshold for the model's confidence to determine when its predictions can be converted into pseudo-labels. However, this method introduces two significant limitations. First, it does not account for the varying levels of difficulty across different classes, resulting in the model generating more pseudo-labels for the easier classes, thus creating a bias toward those classes (Zhang et al., 2022; Wang et al., 2023). Second, the model's confidence tends to be lower during the initial stages of training and increases as training progresses. As a result, this approach starts by generating few pseudo-labeled examples and then gradually produces high-confidence pseudo-labels for almost all samples later on, which can lead to error accumulation and low generalization (Arazo et al., 2020).

These limitations have been addressed by subsequent work in various ways. For instance, FlexMatch (Zhang et al., 2022) and FreeMatch (Wang et al., 2023) propose to adaptively adjust class-wise thresholds based on the estimated learning status of different classes. MarginMatch (Sosea and Caragea, 2023) introduces the Average Pseudo-Margin (APM) score, which tracks the historical predictions for each sample and offers a more reliable confidence estimate. VerifyMatch (Park and Caragea, 2024) uses LLMs to assign initial pseudo-labels to unlabeled data, which are then verified with an SSL model for accuracy. Other works employ the co-training (Blum and Mitchell, 1998) framework. Qiao et al. (2018) aim to alleviate error accumulation by using two distinct networks that are trained simultaneously, each one producing pseudo-labels for the other. Similarly, Sadat and Caragea (2024) employ two distinct networks that are trained simultaneously but instead the networks exchange sample importance weights (not pseudo-labels). JointMatch (Zou and Caragea, 2023) incorporates adaptive local thresholding similar to FreeMatch and introduces a weighting mechanism for disagreement examples, which are shown to be more informative. Multihead Co-training (Chen et al., 2021) addresses the limitation of having to train multiple classifiers by proposing a multihead architecture where each head uses the agreement of the other ones as pseudo-label.

MultiMatch proposes to harness the strengths of both co-training and consistency regularization. While JointMatch has taken initial steps in this direction, the combination of a multihead architecture with a self-adaptive filtering mechanism based on historical predictions remains unexplored.

Semi-supervised Text Classification. SSL has gained significant attention in text classification due to its ability to leverage large-scale unlabeled data alongside limited labeled examples using various approaches. For example, MixText (Chen et al., 2020) mitigates overfitting on limited data by integrating Mixup (Zhang et al., 2018) to interpolate labeled and unlabeled examples in a hidden space. UDA (Xie et al., 2020a) uses strong data augmentation and consistency regularization to enforce

prediction stability across perturbed inputs. AUM-ST (Sosea and Caragea, 2022; Gyawali et al., 2024) refines self-training by filtering unreliable pseudo-labels using the Area Under the Margin (Pleiss et al., 2020), while SAT (Chen et al., 2022) enhances FixMatch (Sohn et al., 2020) by re-ranking weak and strong augmentations based on input similarity. Zou et al. (2023) propose an SSL approach that uses memory banks to handle imbalanced data. Additionally, co-training-based approaches have been explored. For example, Hosseini and Caragea (2023) train two models in parallel – one leveraging unsupervised domain adaptation and the other using SSL in the target domain – allowing them to iteratively refine each other's predictions.

3 MultiMatch

Notation. Let $\mathcal{X} = \{(x_b, y_b); b \in \{1, 2, ..., B\}\}$ be a batch of size B of labeled examples and $\mathcal{U} = \{u_b; b \in \{1, 2, ..., \mu B\}\}$ be a batch of unlabeled examples, where μ is the ratio of labeled to unlabeled data in a batch. Let $\alpha(\cdot)$ and $\mathcal{A}(\cdot)$ be the weak and strong augmentation functions, respectively. The model's predictive distribution for a given input is denoted by $q = p_m(y|\cdot)$ and $\hat{q} = argmax(q)$ is the one-hot encoded pseudo-label. $\mathcal{H}(\cdot, \cdot)$ represents the cross-entropy function.

3.1 Background

FixMatch (Sohn et al., 2020) introduced the most popular recent direction in SSL which consists of combining consistency regularization (Sajjadi et al., 2016) and pseudo-labeling (McLachlan, 1975).

Specifically, for an unlabeled sample u_b, FixMatch creates two augmented versions using weak and strong augmentation functions, $\alpha(\cdot)$ and $\mathcal{A}(\cdot)$ respectively. Both $\alpha(u_b)$ and $\mathcal{A}(u_b)$ are passed through the model to obtain predictions $q_b = p_m(y|\alpha(u_b))$ and $Q_b = p_m(y|\mathcal{A}(u_b))$. The prediction on the weakly-augmented version is converted into a pseudo-label $\hat{q_b} = \arg\max_y(q_b)$. FixMatch uses a predefined threshold τ, such that only the high-confidence pseudo-labels that pass the filtering function $\mathbb{1}_\tau(q_b) = \mathbb{1}(\max(q_b) > \tau)$ are used in the unsupervised loss computation:

$$l_u = \frac{1}{\mu B} \sum_{b=1}^{\mu B} \mathbb{1}_\tau(q_b) H(\hat{q_b}, Q_b) \quad (1)$$

FlexMatch (Zhang et al., 2022) and FreeMatch (Wang et al., 2023) show that using a fixed confidence threshold τ is suboptimal, as it does not account for varying difficulty across classes and for the model's increase in confidence over time. They address these issues by introducing class-wise thresholds, adaptively scaling $\tau(c)$ depending on the learning status of each class. Specifically, FreeMatch computes a self-adaptive global threshold τ_t that reflects the confidence of the model on the whole unlabeled dataset and a self-adaptive local threshold $\tilde{p}_t(c)$ that reflects the confidence of the model for a specific class. Then, the two thresholds are combined in order to obtain the final self-adaptive class thresholds $\tau_t(c)$ as the product between the normalized local thresholds and the global one. The computation of $\tau_t(c)$ is described formally in Appendix C.

These thresholds are used to replace the filtering function from FixMatch with:

$$\mathbb{1}^{(t)}_{Free} = \mathbb{1}(\max(q_b) > \tau_t(\hat{q_b})) \quad (2)$$

MarginMatch (Sosea and Caragea, 2023) builds on the previous approaches by showing that the current confidence of the model is not necessarily a good estimation for the quality of the pseudo-label. Instead, they propose that the historical beliefs should also be taken into account. To this end, they introduce the concept of pseudo-margin (PM) that is computed for each sample with respect to the pseudo-label c at iteration t as the difference between the logit z_c corresponding to the assigned pseudo-label c and the largest other logit:

$$PM^{(t)}_c(u_b) = z^{(t)}_c - \max_{i \neq c} z^{(t)}_i \quad (3)$$

Then, the Average Pseudo-Margin (APM) is computed using Exponential Moving Average using the smoothing parameter λ_m to favor later iterations:

$$APM^{(t)}_c(u_b) = PM^{(t)}_c(u_b) \cdot \frac{\lambda_m}{1+t} + APM^{(t-1)}_c(u_b) \cdot (1 - \frac{\lambda_m}{1+t}) \quad (4)$$

Finally, the APM confidence threshold $\gamma^{(t)}$ is computed for the current iteration using a a newly-introduced virtual class of erroneous examples and the filtering function becomes $\mathbb{1}^{(t)}_{APM} = \mathbb{1}(APM^{(t)}_{\hat{q_b}}(u_b) > \gamma^{(t)})$.

3.2 MultiMatch: Multihead Consistency Regularization Matching

MultiMatch is our proposed semi-supervised learning algorithm that builds upon and extends elements from multiple SSL frameworks. The three

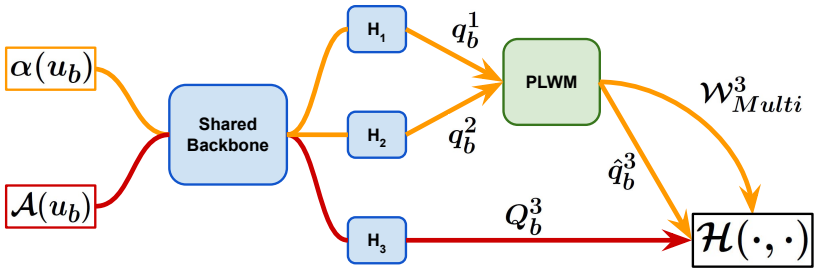


Figure 1: MultiMatch architecture with three heads. Orange and red lines show the path of the weakly augmented and strongly augmented samples, respectively. The green PLWM module receives the q_b^1 and q_b^2 predictions on the weakly augmented samples from heads H_1 and H_2, then generates the pseudo-labels $\hat{q}_b^3$ and the corresponding weights $\mathcal{W}_{Multi}^3$. They are used together with the prediction Q_b^3 of head H_3 on the strongly augmented samples for computing the unsupervised loss $\mathcal{L}_u^3$ using the cross-entropy function. While the diagram shows the unlabeled loss computation for head H_3, similar computations are used for H_1 and H_2.

primary sources of inspiration are **Multi**head Co-training (Chen et al., 2021), Margin**Match** (Sosea and Caragea, 2023), and Free**Match** (Wang et al., 2023), which serve as representative approaches for the co-training and the consistency regularization with pseudo-labeling paradigms, respectively.

The pipeline for training a model with MultiMatch is shown in Figure 1. Each batch of unlabeled examples is augmented using the weak and strong augmentation functions $\alpha(\cdot)$ and $\mathcal{A}(\cdot)$, respectively. The two augmented versions are forwarded through the model to make predictions. Note that, similar to Multihead Co-training, the model is multiheaded, which enables the co-training functionalities, while the shared backbone preserves the inference time of a single model. We use three heads in our presentation of the algorithm, both for simplicity and because Chen et al. (2021) identified this to be the optimal choice.

As multiple predictions are obtained, pseudo-label selection is less straightforward than in classical pseudo-labeling, unless disagreement samples are discarded – a strategy we consider suboptimal. To address this, we introduce a pseudo-label weighting module (PLWM), which serves three key functions: selecting pseudo-labels, filtering out unreliable pseudo-labels and weighting samples according to their estimated difficulty. These functions are achieved through our three-fold approach, which integrates and refines head agreements, self-adaptive thresholding, and APM scores.

The pseudocode of MultiMatch can be viewed in Algorithm 1. More details about the algorithm are provided in the remainder of this section.

3.3 Pseudo-Label Weighting Module (PLWM)

In the co-training framework, the training samples are usually part of two categories: the ones where the models agree on the prediction and the disagreement samples. However, different methods treat them in different ways. On one hand, some methods consider the agreement samples to be irrelevant, as they do not convey new information, so they are removed from the loss computation (Yu et al., 2019) to prevent the convergence of the models. These methods only use the disagreement samples for training. On the other hand, the multihead methods consider the agreement samples to provide high-confidence pseudo-labels, so they use them and remove the disagreement samples from the loss computation (Chen et al., 2021). Finally, JointMatch (Zou and Caragea, 2023) tries to balance the two sides by using the agreement only as a weighting mechanism, while the filtering itself is done by a distinct confidence-based heuristic.

We introduce a new mechanism of using the agreement of the heads. By combining it with the APM score, we split the generated pseudo-labels into 3 categories: *not useful*, *useful & difficult*, and *useful & easy*.

If the heads of the model used to generate the pseudo-label reach agreement ($\hat{q}_b^i = \hat{q}_b^j$) and both have high confidence in terms of APM score, the sample is considered *useful & easy*, so it is included in the loss computation with weight 1, using the agreed pseudo-label. If one of the heads has high confidence and the other one has low confidence, in terms of APM score, we consider that sample to be *useful & difficult*, as one head is not confident in the prediction. In this case, the selected pseudo-label is the one generated by the confident head. We include it in the loss computation with a larger weight w_d. If both heads have low confidence or if both heads have high confidence but disagree on the label, we consider that sample to be *not useful* for the current iteration and assign it weight 0. This is equivalent to filtering out the sample.

Algorithm 1 MultiMatch

Require: Labeled batch $\mathcal{X} = \{(x_b, y_b); b \in (1, 2, ..., B)\}$, unlabeled batch $\mathcal{U} = \{u_b; b = (1, 2, ..., \mu B)\}$, where B is the batch size of labeled data, μ is the ratio of unlabeled to labeled data, unsupervised loss weight w_u, disagreement weight δ, FreeMatch EMA decay λ_f, MarginMatch EMA decay λ_m, weak and strong augmentations $\alpha(\cdot)$, $\mathcal{A}(\cdot)$.

for h in 1 to 3 **do** ▷ For each classification head.

  $i, j = \{1, 2, 3\} \setminus \{h\}$ ▷ The other 2 heads are used for generating pseudo-labels.

  $\mathcal{L}_s^h = \frac{1}{B} \sum_{b=1}^{B} \mathcal{H}(p_b, p_m^h(y|\alpha(x_b)))$ ▷ Compute the supervised loss for head h using the labeled data.

  Compute $\mathbb{1}_{Free}^{(t)}(i)$ and $\mathbb{1}_{Free}^{(t)}(j)$ using Eq. 2 for the remaining heads i and j.

  Compute $APM_{k,c}^{(t)}(u_b)$ for all classes $c \in \{1, ..., C\}$, heads $k \in \{i, j\}$ and all samples $u_b \in \mathcal{U}$ using Eq. 3 and 4.

  Select $\hat{q}_b^h$ and compute W_{Multi}^h using Eq. 5. ▷ Use PLWM to select, filter, and weight pseudo-labels.

  $\mathcal{L}_{u,t}^h = \frac{1}{\mu B} \sum_{b=1}^{\mu B} W_{Multi}^h \cdot H(\hat{q}_b^h, Q_b^h)$ ▷ Compute the unsupervised loss for head h using the generated pseudo-labels and weights.

end for

Compute $\gamma_{k,c}^{(t)}$, $k \in \{1, 2, 3\}$, using Eq. 6. ▷ Update our self-adaptive APM thresholds.

$\mathcal{L}_s = \sum_{k \in \{1,2,3\}} \mathcal{L}_s^k$ and $\mathcal{L}_u = \sum_{k \in \{1,2,3\}} \mathcal{L}_u^k$

$\mathcal{L} = \mathcal{L}_s + w_u \cdot \mathcal{L}_u$ ▷ Compute the total loss.

To define high confidence in terms of the APM score, we introduce a novel method for computing self-adaptive APM thresholds, which is detailed in Section 3.4. Finally, we employ an additional self-adaptive filtering based on the current confidence of each head. The computation of the weights W_{Multi}^h for the head h and the unlabeled batch u_b is described formally in Eq. 5.

$$W_{Multi}^h = [1 \cdot (\mathbb{1}_{Multi}^i \wedge \mathbb{1}_{Multi}^j \wedge \mathbb{1}_{Agree}^h) + \\ + w_d \cdot (\mathbb{1}_{Multi}^i \oplus \mathbb{1}_{Multi}^j)] \cdot \mathbb{1}_{FreeMulti}^h \quad (5)$$

where $\mathbb{1}_{Multi}^k = \mathbb{1}(APM_{\hat{q}_b^k}^{(t)}(u_b) > \gamma_{k,c}^{(t-1)})$, $k \in \{i, j\}$ marks the high confidence of head k, $\mathbb{1}_{Agree}^h = [\hat{q}_b^i = \hat{q}_b^j]$, $\{i, j\} = \{1, 2, 3\} \setminus \{h\}$ is the agreement of the remaining heads i and j, and $\mathbb{1}_{FreeMulti}^h = \mathbb{1}_{Free}^i \vee \mathbb{1}_{Free}^j$ is the self-adaptive threshold filtering based on the current confidence, adapted to multiple heads.

3.4 Self-adaptive APM Thresholds

The use of Average-Pseudo Margins (Sosea and Caragea, 2023) has shown great potential in addressing the common issues that come with relying on the current model's confidence for evaluating the quality of pseudo-labels. However, the thresholding approach used by MarginMatch still has its limits, as the adaptive threshold $\gamma^{(t)}$ is the same for all classes. We address this by using class-wise thresholds that take into account the different learning difficulty of each class.

Furthermore, we take advantage of our multi-head architecture and instead of introducing an additional virtual class, we use the subset of examples where the heads reached agreement as the subset used for the threshold estimation.

By starting from a subset assumed to be correctly pseudo-labeled and choosing the $f = 5^{th}$ percentile, we obtain a threshold estimate that favors correctly pseudo-labeled samples, minimizing the number of incorrect pseudo-labels that pass the threshold. On the other hand, MarginMatch starts from an incorrectly classified subset and uses the 95th percentile value, thus potentially choosing a threshold that allows for more incorrect pseudo-labels to pass, leading to error accumulation for the model. Finally, adhering to our principle of minimizing the noisy pseudo-labels, we set a lower bound $\gamma_{min} = 0$ for the computed threshold.

Eq. 6 shows the formula for choosing the threshold $\gamma_{h,c}^{(t)}$ for head h and class c at iteration t by using the agreement of the other heads $k \neq h$:

$$\gamma_{h,c}^{(t)} = max(\gamma_{min}, \text{percentile}_f(\\ \cup_{k \neq h} \{APM_{k,c}^{(t)}(u_b) \mid \mathbb{1}_{Agree}^{h,c}\})) \quad (6)$$

where $\mathbb{1}_{Agree}^{h,c} = [\hat{q}_b^i = \hat{q}_b^j = c]$, $\{i, j\} = \{1, 2, 3\} \setminus \{h\}$ marks if the remaining two heads have agreed on the pseudo-label c.

4 Experiments

We assess the performance of our MultiMatch algorithm across a diverse set of SSL benchmark datasets. Specifically, we utilize the USB benchmark (Wang et al., 2022), which offers a stan-

Dataset	IMDB		AG News		Amazon Review		Yahoo! Answer		Yelp Review		Mean	Fried.	Final
# Label	20	100	40	200	250	1000	500	2000	250	1000	error	rank	rank
Supervised (Full)	5.69±0.15	5.72±0.13	5.78±0.07	5.73±0.11	36.40±0.05	36.40±0.05	24.87±0.04	24.84±0.04	32.04±0.03	32.04±0.03	20.95	-	-
Supervised (Small)	20.31±2.79	14.02±1.22	15.06±1.08	14.25±0.97	52.31±1.28	47.53±0.69	37.43±0.29	33.26±0.10	51.22±0.98	46.71±0.37	33.21	-	-
AdaMatch	8.09±0.99	7.11±0.20	11.73±0.17	11.22±0.95	46.72±0.72	42.27±0.25	32.75±0.35	30.44±0.31	45.40±0.96	40.16±0.49	27.59	6.5	5
FixMatch	7.72±0.33	7.33±0.13	30.17±1.87	11.71±1.95	47.61±0.83	43.05±0.54	33.03±0.49	30.51±0.53	46.52±0.94	40.65±0.46	29.83	10.2	11
FlexMatch	7.82±0.77	7.41±0.38	16.38±3.94	12.08±0.73	45.73±1.60	42.25±0.33	35.61±1.08	31.13±0.18	43.35±0.69	40.51±0.34	28.23	9.4	9
Dash	8.34±0.86	7.55±0.35	31.67±13.1	13.76±1.67	47.10±0.74	43.09±0.60	35.26±0.33	31.19±0.29	45.24±2.02	40.14±0.79	30.33	11.7	13
CRMatch	8.96±0.88	7.16±0.09	12.28±1.43	11.08±1.24	45.49±0.98	43.07±0.50	32.51±0.40	29.98±0.07	45.71±0.63	40.62±0.28	27.69	7.6	6
CoMatch	7.44±0.30	7.72±1.14	11.95±0.76	10.75±0.35	48.76±0.90	43.36±0.21	33.48±0.51	30.25±0.35	45.40±1.12	40.27±0.51	27.94	8.1	8
SimMatch	7.93±0.55	7.08±0.33	14.26±1.51	12.45±1.37	45.91±0.95	42.21±0.30	33.06±0.20	30.16±0.21	46.12±0.48	40.26±0.62	27.94	7.6	6
FreeMatch	8.94±0.21	7.95±0.45	12.98±0.58	11.73±0.63	46.41±0.60	42.64±0.06	32.77±0.26	30.32±0.18	47.95±1.45	40.37±1.00	28.21	9.7	10
SoftMatch	7.76±0.58	7.97±0.72	11.90±0.27	11.72±1.58	45.29±0.95	42.21±0.20	33.07±0.31	30.44±0.62	44.09±0.50	39.76±0.13	27.42	6.4	4
Multihead Co-training	8.70±0.88	7.46±0.68	22.72±6.09	13.48±1.51	46.22±0.70	43.07±0.79	35.17±1.58	30.81±0.28	46.46±1.28	40.79±0.49	29.49	12.2	14
MarginMatch	7.19±0.39	6.99±0.19	**10.65±0.19**	11.03±0.99	44.81±1.23	42.14±0.67	32.08±0.70	29.55±0.15	42.93±1.48	39.13±0.34	26.65	2.3	2
SequenceMatch	12.99±7.64	8.34±0.57	16.17±2.53	12.56±0.87	47.97±2.39	42.58±0.21	34.92±0.78	30.28±0.28	45.60±0.84	40.44±0.51	29.19	11.3	12
CGMatch	7.07±0.36	**6.79±0.10**	11.95±0.63	11.29±0.47	44.77±1.39	42.61±0.47	32.15±0.58	29.85±0.16	44.34±0.26	40.14±0.07	27.10	3.9	3
MultiMatch	**6.89±0.07**	6.98±0.27	11.14±0.96	**10.59±0.66**	**44.43±0.98**	**42.09±0.28**	**30.90±0.70**	**29.39±0.39**	**42.16±0.79**	**39.08±0.55**	**26.37**	**1.2**	**1**

Table 1: Test error rates on IMDB, AG News, Amazon Review, Yahoo! Answer, and Yelp Review datasets using two setups with different sizes for the labeled set. The best result for each setup is highlighted in **blue**. The full table is shown in Appendix G.

dardized evaluation framework for semi-supervised classification across vision, language, and audio tasks. We perform experiments with various numbers of labeled examples on the natural language processing (NLP) datasets included in USB: IMDB (Maas et al., 2011), AG News (Zhang et al., 2015), Amazon Review (McAuley and Leskovec, 2013), Yahoo! Answer (Chang et al., 2008) and Yelp Review (Asghar, 2016). For a straightforward and fair comparison with the 16 SSL algorithms already included in USB, we maintain their original experimental setup without modifications. This ensures consistency across evaluations and includes using the uncased version of BERT-Base (Devlin, 2018) for the shared backbone, preserving the train-validation-test splits, the number of labeled and unlabeled examples per class, and all training hyperparameters. We provide dataset statistics and split information in Appendix E, and more details on the experimental setup in Appendix D.

For the supervised baselines and the SSL baselines implemented in USB, we report the results from the USB GitHub page[1], which provide an improvement over those originally reported by Wang et al. (2022) by using a larger batch size. For the other 4 baselines (Multihead Co-training, MarginMatch, SequenceMatch, CGMatch) and for MultiMatch we add the implementations to the USB framework and report the average test error of 3 runs per experiment (using the same seeds). In addition to the mean error rate across tasks, we compute the Friedman rank of each algorithm as the mean rank over all tasks $\text{rank}_F = \frac{1}{m}\sum_{i=1}^{m} \text{rank}_i$, where $m = 10$ is the number of evaluation setups and rank_i is the rank of the algorithm in the i-th setup.

[1] https://github.com/microsoft/Semi-supervised-learning

Imbalanced settings. While the research on semi-supervised learning in NLP widely uses balanced setups for evaluation, the class distributions of many real-world text classification datasets are imbalanced (Gui et al., 2024). Moreover, while it is well known that classifiers trained on imbalanced data have a bias toward the majority classes, this issue is even more problematic for SSL algorithms where the use of biased pseudo-labels increases the bias over time (Kim et al., 2020).

In order to assess the robustness of MultiMatch when data is imbalanced, we draw inspiration from the setup used by ABC in Computer Vision (Lee et al., 2021) to create class-imbalanced versions of the text datasets from USB. As we focus on high-imbalance scenarios, we consider a long-tailed imbalance, where the number of data points decreases exponentially from the largest to the smallest class. Formally, the number of samples from class c is $N_c = N_1 \cdot \gamma^{-\frac{c-1}{C-1}}$, where C is the number of classes, $c \in \{1, ..., C\}$, and $\gamma = \frac{N_1}{N_C}$ is the imbalance factor between the largest and the smallest class. For each dataset, we consider two highly imbalanced setups: first, we consider a long-tail imbalance that is similar in the labeled and unlabeled datasets. In this case, we set the imbalance factor $\gamma = 100$, the size of the largest class $N_1 = 1000$ and of the smallest class $N_C = 10$ and the unlabeled set to be 10 times larger than the labeled one. For the second setup, we keep the same imbalance for the labeled dataset, but we use an unlabeled imbalance factor of $\gamma = -100$, which represents a long-tail distribution where the most represented classes in the labeled dataset are the least represented ones in the unlabeled set. This represents an extreme case where the unlabeled corpus is col-

Dataset	IMDB		AG News		Amazon Review		Yahoo! Answer		Yelp Review		Mean	Fried.	Final
Imbalance	100	-100	100	-100	100	-100	100	-100	100	-100	error	rank	rank
FixMatch	49.95±0.06	49.92±0.14	31.95±5.30	36.09±6.50	64.69±0.92	61.93±4.41	51.64±0.75	52.78±1.45	65.57±3.19	57.43±6.22	52.20	8.4	10
+ ABC	49.69±0.43	45.90±7.07	38.08±8.80	21.21±5.72	62.02±1.03	56.27±1.16	61.15±3.12	53.79±9.31	62.63±0.89	58.37±6.97	50.91	6.3	7
FreeMatch	49.95±0.03	42.75±12.56	26.55±4.87	**24.58±3.71**	62.26±0.48	59.22±1.94	44.93±1.00	**40.96±0.43**	63.92±3.98	59.13±1.08	47.43	6.0	6
+ ABC	49.97±0.04	30.62±18.18	23.76±5.45	24.01±1.66	58.13±0.93	**54.86±0.34**	49.60±7.68	44.04±4.18	61.58±2.69	54.02±3.44	45.06	3.9	4
Multihead Co-training	49.91±0.12	50.00±0.00	30.88±2.81	34.43±4.51	62.90±2.24	**58.05±4.53**	52.05±1.28	51.39±1.51	64.32±3.06	58.62±3.34	51.25	7.9	9
+ ABC	49.76±0.24	50.00±0.00	25.29±1.41	26.18±1.20	**57.99±3.54**	57.07±1.56	46.92±2.81	49.00±6.97	63.48±3.90	55.38±1.50	48.11	5.1	5
MarginMatch	49.60±0.62	49.99±0.01	26.46±0.99	33.85±2.93	64.74±0.39	66.45±0.81	50.59±0.86	53.96±0.81	63.33±0.58	63.70±2.41	52.27	7.8	8
+ ABC	**45.04±4.14**	42.55±12.85	**23.74±2.77**	**17.88±0.75**	59.98±1.45	55.71±3.13	48.48±9.07	52.75±2.71	61.43±3.15	55.42±0.33	46.30	3.2	2
MultiMatch	**49.17±0.88**	**26.05±6.28**	**21.11±0.49**	25.36±2.60	**61.03±1.64**	59.66±2.58	**41.01±0.79**	41.46±4.34	**60.14±3.41**	**56.71±1.87**	**44.17**	**3.3**	**3**
+ ABC	48.81±1.98	**17.21±3.78**	29.36±11.90	19.65±4.39	62.13±1.46	57.12±0.88	**41.05±1.34**	**42.72±2.57**	**60.71±2.68**	**53.87±3.03**	**43.26**	**3.1**	**1**

Table 2: Test error rates in imbalanced setups. The best results **without**/**with** ABC are highlighted in **blue**/**purple**. Imbalance 100: similar distributions for labeled and unlabeled data; Imbalance -100: reversed long-tail distribution for the unlabeled set.

lected separately from the labeled dataset and the class distributions might be entirely different.

Since SSL algorithms exhibit significantly different performance in imbalanced settings compared to balanced ones, and the performance gap between algorithms is larger in imbalanced setups, we integrate a class-imbalance semi-supervised learning (CISSL) method, ABC (Lee et al., 2021), to assess the potential of bridging this gap. ABC accounts for class imbalance by attaching an additional balanced classifier, without discarding any data. Given its seamless integration with existing SSL algorithms, we conduct additional experiments by enhancing each tested SSL algorithm with ABC in imbalanced settings. Further details on CISSL and ABC are provided in the Appendix A.

5 Results

The comparison between MultiMatch and other SSL methods on the five text classification datasets is presented in Table 1. Our results show that MultiMatch consistently outperforms all the baselines, achieving top performance in 8 out of 10 setups and ranking first according to the Friedman test among 21 SSL algorithms. While the overall mean error also decreases by an absolute value of 0.28% compared to the second best approach, we notice that the improvement is higher when the number of labeled examples is lower. Specifically, the improvement over MarginMatch is on average 0.43% in the lower-resource setting, compared to 0.14% when more labeled data is provided.

By analyzing the baselines that inspired our approach, we observe that MarginMatch consistently outperforms all methods except MultiMatch, reinforcing the importance of leveraging historical evidence for pseudo-labels filtering. In contrast, Multihead Co-training and FreeMatch have a lower rank, 14 and 10, respectively, indicating that the voting mechanism and the self-adaptive thresholds, while beneficial, are not among the most impactful techniques when used in isolation. However, when refined and incorporated into our unique three-fold pseudo-label weighting module (PLWM), their combined effect leads to substantial performance gains across all evaluation setups.

Imbalanced settings. We evaluate MultiMatch and representatives of each incorporated technique in newly created highly imbalanced setups of the same datasets from USB. Specifically, as shown in Table 2, we compare MultiMatch with MarginMatch, representing the Average Pseudo-Margins approach; Multihead Co-training, embodying the voting mechanism from co-training; FreeMatch, representing the self-adaptive class thresholds technique; and FixMatch, serving as a baseline for pseudo-labeling with consistency regularization.

MultiMatch outperforms all the baselines, ranking first in 7 out of 10 setups and outperforming the second-best approach, FreeMatch, by 3.26% averaged across all tasks. In particular, MultiMatch reaches state-of-the-art performance in all five setups with the imbalance factor 100, when the unlabeled distribution is similar to the labeled one, and surpasses FreeMatch by 3.48% when the unlabeled distribution is different. Comparing to MarginMatch, the main competitor in the balanced setup, MultiMatch surpasses it by an average of 8.1% in the imbalanced setting, with an impressive 11.74% improvement when the unlabeled distribution differs from the labeled one.

The superior performance of MultiMatch in highly imbalanced settings can be attributed to several CISSL techniques incorporated into its design. First, the use of class-wise thresholds enables the algorithm to account for the distinct learning progress of individual classes. For the same reason, FreeMatch shows a substantial improvement in im-

Dataset	IMDB		AG News		Amazon Review		Yahoo! Answer		Yelp Review		Mean	Fried.	Final
# Label	20	100	40	200	250	1000	500	2000	250	1000	error	rank	rank
MultiMatch	6.89±0.07	**6.98±0.27**	11.14±0.96	10.59±0.66	44.43±0.98	42.09±0.28	**30.90±0.70**	**29.39±0.39**	**42.16±0.79**	**39.08±0.55**	**26.37**	**1.8**	**1**
— w_d	7.08±0.32	7.01±0.31	**10.93±0.43**	10.73±0.62	44.47±1.11	42.17±0.18	31.24±0.47	29.64±0.51	44.06±1.28	39.41±0.46	26.67	3.7	4
— $w_d = 0$	7.19±0.29	7.20±0.17	11.80±0.88	10.53±0.27	44.19±0.79	42.13±0.27	31.12±0.75	29.63±0.31	43.20±1.24	39.61±0.14	26.66	3.3	3
— $\mathbb{1}_{FreeMulti}$	7.23±0.18	7.02±0.24	18.41±5.86	11.28±0.86	44.58±0.96	42.08±0.29	36.15±1.36	30.35±0.70	43.94±1.68	39.16±0.36	28.02	4.6	5
— $\mathbb{1}_{Multi}$	**6.88±0.14**	7.46±0.50	15.77±3.04	11.66±0.18	44.57±0.80	42.31±0.36	32.13±0.45	30.35±0.26	44.68±0.89	39.99±0.22	27.58	5.2	6
— γ_{min}	7.09±0.24	**6.98±0.20**	11.18±0.66	**10.56±0.63**	**44.11±0.79**	**41.99±0.18**	31.14±0.61	29.59±0.27	43.37±0.65	39.17±0.31	26.52	2.4	2

Table 3: Ablation study by removing one component from MultiMatch. The best results are highlighted in **blue**.

balanced setups, ranking second before the ABC enhancement. Second, the use of percentile-based thresholds mitigates imbalance amplification, as shown by Guo and Li (2022). Finally, MultiMatch explicitly promotes lower pseudo-label impurity through several mechanisms. The use of multiple filters increases the difficulty for incorrect pseudo-labels to pass, while the subset employed for threshold computation results in generally higher thresholds compared to MarginMatch. This approach helps mitigate long-term error accumulation, a key challenge in imbalanced SSL. For a more detailed technical discussion, see Appendix B.

Enhancing SSL algorithms with ABC improves the overall performance of each algorithm. Although MarginMatch benefits the most with a 5.97% improvement and MultiMatch the least with only 0.91%, MultiMatch remains the best-performing approach, achieving state-of-the-art results on five setups and an error rate 1.8% lower than FreeMatch+ABC, the second-best ABC-enhanced method. Notably, even without ABC enhancement, MultiMatch still outperforms all other ABC-enhanced algorithms in terms of mean error. This highlights the robustness of our approach to imbalanced class distributions, a trait that is essential for real-world text classification datasets.

6 Ablation Study and Analysis

To demonstrate the effectiveness of each component in MultiMatch, we evaluate its performance after systematically removing different elements, as presented in Table 3. We observe a decline in performance after the removal of each, indicating that all components in MultiMatch play a significant role in achieving optimal performance.

First, in order to assess the importance of *useful & difficult* examples, we remove the weight w_d, so they will be weighted similar to the *useful & easy* samples. This leads to a drop in performance on 9 out of 10 setups, with a mean error rate increase of 0.3%. Similarly, by completely removing these examples from the training data, on average we

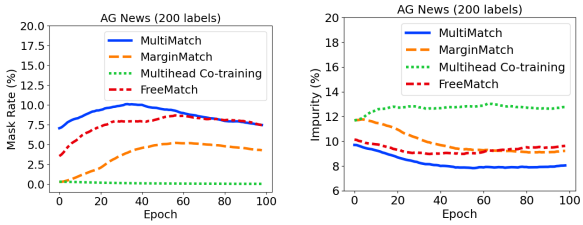


Figure 2: Mask rate and impurity on AG News with 200 labels.

do not notice any substantial differences. Second, we remove the filtering based on the self-adaptive thresholds that rely on the current confidence of the model, $\mathbb{1}_{FreeMulti}$. This results in our PLWM using only the heads agreement and the APM score for filtering pseudo-labels. This leads to a significant drop in performance of 1.65%, highlighting the importance of current model's beliefs in our PLWM. Third, we remove the APM-based filtering $\mathbb{1}_{Multi}$ from MultiMatch, obtaining a PLWM module that only uses the agreement samples where at least one head has high current confidence in the selected pseudo-label. In this case, the mean error rate increases by 1.21%, and the Friedman test places it the lowest among all the setups in the ablation study. Finally, removing the low limit for the class-wise thresholds (γ_{min}) has the lowest impact of only 0.15% error rate, but it still provides additional proof that a stricter filtering mechanism is essential for reducing the noise in the pseudo-labels and preventing their degradation over time.

Pseudo-Label Quality Analysis. We compare MultiMatch with MarginMatch, Multihead Co-training, and FreeMatch in terms of pseudo-label quality using two metrics: *mask rate* and *impurity*. We show the results on the AG News task with 200 labels in Figure 2 (similar results are shown in Figure 3 in Appendix B for the imbalanced setting). The *mask rate* is defined as the proportion of pseudo-labeled examples that were excluded from training at epoch t, while *impurity* is the proportion of pseudo-labeled examples that participate in training at epoch t but have an incorrect label. As expected, our PLWM module employs a stricter filtering mechanism, resulting in a higher mask

Task	Humanitarian				Informative			
	Acc	P	R	F1	Acc	P	R	F1
MMBT Supervised (Kiela et al., 2019)	86.71	87.20	86.75	86.74	89.44	90.07	90.06	89.87
FixMatch (Sohn et al., 2020)	88.55	88.87	88.59	88.51	89.96	89.91	90.00	89.91
FixMatch LS (Sirbu et al., 2022)	88.66	89.04	88.70	88.74	90.38	90.35	90.42	90.36
FreeMatch (Wang et al., 2023)	84.88	86.93	84.93	85.54	90.35	90.52	90.39	90.43
Multihead Co-training (Chen et al., 2021)	88.34	88.68	88.38	88.28	90.68	90.66	90.71	90.67
MarginMatch (Sosea and Caragea, 2023)	87.39	88.48	87.43	87.54	89.86	90.12	89.86	89.94
MultiMatch	**89.18**	**89.58**	**89.18**	**89.14**	**91.36**	**91.37**	**91.36**	**91.37**

Table 4: Classification results on the multimodal *Humanitarian* and *Informative* CrisisMMD tasks. The best result for each metric is highlighted in **blue**. **Acc** represents the accuracy, while **P**, **R** and **F1** are the weighted precision, recall and F1, respectively.

rate than all other approaches. In contrast, relying only on heads agreement proves ineffective, as Multihead Co-training exhibits a mask rate of less than 1%. Meanwhile, our strict filtering, combined with a targeted approach for handling *difficult* examples, effectively minimizes impurity in both setups. In contrast, Multihead Co-training incorporates many erroneous pseudo-labels because of its weak filtering, while the self-adaptive thresholds of FreeMatch fail to prevent error accumulation in the imbalanced setup, causing impurity to rise.

7 Performance in Multimodal Settings

We evaluate the generalizability of MultiMatch in real-world multimodal settings by applying it to the multimodal version of the CrisisMMD dataset (Ofli et al., 2020) and compare its performance against several representative baselines. We adopt the experimental setup from Sirbu et al. (2022), which includes access to unlabeled data suitable for SSL. Specifically, we use MMBT (Kiela et al., 2019) as the supervised backbone and follow the same hyperparameter configuration as in their work. Importantly, we apply MultiMatch with the same hyperparameters used in the NLP experiments, without any task-specific tuning.

We selected CrisisMMD for several reasons. First, it provides real-world data from the crisis response domain. Second, the dataset is multimodal – text and images – unlike the previously used NLP datasets. Third, it supports multiple classification tasks with distinct characteristics: for example, the Informative task features a nearly balanced binary classification setup, while the Humanitarian task involves five classes and is highly imbalanced. Additional data details can be found in Ofli et al. (2020).

As shown in Table 4, MultiMatch demonstrates strong generalization, consistently outperforming all other SSL baselines across both tasks. In contrast, other methods exhibit inconsistent performance. For instance, FreeMatch and Multihead Co-training perform well on the Informative task but struggle on the Humanitarian task. MarginMatch, which previously showed competitive results on NLP datasets, is outperformed by several alternatives in this multimodal setting. While we do not delve into a deeper investigation of domain-specific challenges, the results highlight the robustness of MultiMatch in multimodal, domain-specific, and highly imbalanced setups.

8 Conclusion

In this work, we introduced MultiMatch, a novel semi-supervised learning algorithm that integrates co-training, consistency regularization, and pseudo-labeling into a unified framework. Our approach effectively balances pseudo-label selection, filtering, and weighting, enhancing both robustness and performance. Extensive experiments on the USB benchmark demonstrate that MultiMatch consistently outperforms 18 baseline methods in both balanced and highly imbalanced settings, which are crucial for real-world text classification tasks. Furthermore, our ablation study highlights the significance of each component in MultiMatch, demonstrating that our strict three-fold filtering mechanism is crucial for minimizing erroneous pseudo-labels and enhancing performance. These results emphasize the effectiveness of hybrid SSL strategies and open new directions for developing more adaptive and robust techniques.

Acknowledgements

This research is supported by the project "Romanian Hub for Artificial Intelligence - HRIA", Smart Growth, Digitization and Financial Instruments Program, 2021-2027, MySMIS no. 334906; the NSF IIS award 2107518; and a UIC Discovery Partners Institute (DPI) award. Any opinions, findings, and conclusions expressed here are those of the authors and do not necessarily reflect the views of NSF or DPI. We also thank our anonymous reviewers for their insightful feedback and suggestions.

Limitations

While MultiMatch demonstrates strong performance across various setups, it has certain limitations. First, its reliance on multiple classification heads increases computational overhead compared to single-head SSL methods. In our case, the number of parameters increased by roughly 1% (from 110M to 111M), but this may vary depending on the backbone. However, this overhead is negligible compared to traditional co-training approaches that train two distinct models. We extend the efficiency considerations in Appendix F. Second, applying MultiMatch to specialized applications may require domain-specific knowledge integration (e.g., customized augmentation strategies) and additional hyperparameter tuning. However, this limitation is shared by all SSL methods. In Section 7, we take initial steps toward assessing the generalizability of MultiMatch by applying it in a multimodal, real-world experimental setting. The results highlight the particular robustness of MultiMatch across two distinct tasks, even without any hyperparameter tuning, in contrast to other baseline methods. Nonetheless, further analysis remains advisable for highly sensitive applications.

References

Eric Arazo, Diego Ortego, Paul Albert, Noel E O'Connor, and Kevin McGuinness. 2020. Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *2020 International joint conference on neural networks (IJCNN)*, pages 1–8. IEEE.

Nabiha Asghar. 2016. Yelp dataset challenge: Review rating prediction. *arXiv preprint arXiv:1605.05362*.

David Berthelot, Nicholas Carlini, Ekin D Cubuk, Alex Kurakin, Kihyuk Sohn, Han Zhang, and Colin Raffel. 2019a. Remixmatch: Semi-supervised learning with distribution alignment and augmentation anchoring. *arXiv preprint arXiv:1911.09785*.

David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin A Raffel. 2019b. Mixmatch: A holistic approach to semi-supervised learning. *Advances in neural information processing systems*, 32.

David Berthelot, Rebecca Roelofs, Kihyuk Sohn, Nicholas Carlini, and Alex Kurakin. 2021. Adamatch: A unified approach to semi-supervised learning and domain adaptation. *arXiv preprint arXiv:2106.04732*.

Avrim Blum and Tom Mitchell. 1998. Combining labeled and unlabeled data with co-training. In *Proceedings of the eleventh annual conference on Computational learning theory*, pages 92–100.

Ming-Wei Chang, Lev-Arie Ratinov, Dan Roth, and Vivek Srikumar. 2008. Importance of semantic representation: Dataless classification. In *Aaai*, volume 2, pages 830–835.

Hao Chen, Ran Tao, Yue Fan, Yidong Wang, Jindong Wang, Bernt Schiele, Xing Xie, Bhiksha Raj, and Marios Savvides. 2023. Softmatch: Addressing the quantity-quality trade-off in semi-supervised learning. *arXiv preprint arXiv:2301.10921*.

Hui Chen, Wei Han, and Soujanya Poria. 2022. Sat: Improving semi-supervised text classification with simple instance-adaptive self-training. *arXiv preprint arXiv:2210.12653*.

Jiaao Chen, Zichao Yang, and Diyi Yang. 2020. Mixtext: Linguistically-informed interpolation of hidden space for semi-supervised text classification. *Preprint*, arXiv:2004.12239.

Mingcai Chen, Yuntao Du, Yi Zhang, Shuwei Qian, and Chongjun Wang. 2021. Semi-supervised learning with multi-head co-training. *Preprint*, arXiv:2107.04795.

Bo Cheng, Jueqing Lu, Yuan Tian, Haifeng Zhao, Yi Chang, and Lan Du. 2025. Cgmatch: A different perspective of semi-supervised learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 15381–15391.

Janez Demšar. 2006. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, 7(Jan):1–30.

Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Yue Fan, Anna Kukleva, Dengxin Dai, and Bernt Schiele. 2023. Revisiting consistency regularization for semi-supervised learning. *International Journal of Computer Vision*, 131(3):626–643.

Qian Gui, Hong Zhou, Na Guo, and Baoning Niu. 2024. A survey of class-imbalanced semi-supervised learning. *Machine Learning*, 113(8):5057–5086.

Lan-Zhe Guo and Yu-Feng Li. 2022. Class-imbalanced semi-supervised learning with adaptive thresholding. In *International Conference on Machine Learning*, pages 8082–8094. PMLR.

Nikesh Gyawali, Iustin Sirbu, Tiberiu Sosea, Sarthak Khanal, Doina Caragea, Traian Rebedea, and Cornelia Caragea. 2024. GunStance: Stance detection for gun control and gun regulation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12027–12044, Bangkok, Thailand. Association for Computational Linguistics.

Mahshid Hosseini and Cornelia Caragea. 2023. Semi-supervised domain adaptation for emotion-related tasks. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 5402–5410.

Douwe Kiela, Suvrat Bhooshan, Hamed Firooz, and Davide Testuggine. 2019. Supervised multimodal bitransformers for classifying images and text. *arXiv preprint arXiv:1909.02950*.

Jaehyung Kim, Youngbum Hur, Sejun Park, Eunho Yang, Sung Ju Hwang, and Jinwoo Shin. 2020. Distribution aligning refinery of pseudo-label for imbalanced semi-supervised learning. *Advances in neural information processing systems*, 33:14567–14579.

Dong-Hyun Lee et al. 2013. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *Workshop on challenges in representation learning, ICML*, volume 3, page 896. Atlanta.

Hyuck Lee, Seungjae Shin, and Heeyoung Kim. 2021. Abc: Auxiliary balanced classifier for class-imbalanced semi-supervised learning. *Advances in Neural Information Processing Systems*, 34:7082–7094.

Junnan Li, Caiming Xiong, and Steven CH Hoi. 2021. Comatch: Semi-supervised learning with contrastive graph regularization. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9475–9484.

Andrew Maas, Raymond E Daly, Peter T Pham, Dan Huang, Andrew Y Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, pages 142–150.

Julian McAuley and Jure Leskovec. 2013. Hidden factors and hidden topics: understanding rating dimensions with review text. In *Proceedings of the 7th ACM conference on Recommender systems*, pages 165–172.

Geoffrey J McLachlan. 1975. Iterative reclassification procedure for constructing an asymptotically optimal rule of allocation in discriminant analysis. *Journal of the American Statistical Association*, 70(350):365–369.

Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, and Shin Ishii. 2018. Virtual adversarial training: a regularization method for supervised and semi-supervised learning. *IEEE transactions on pattern analysis and machine intelligence*, 41(8):1979–1993.

Khanh-Binh Nguyen. 2024. Sequencematch: Revisiting the design of weak-strong augmentations for semi-supervised learning. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 96–106.

Ferda Ofli, Firoj Alam, and Muhammad Imran. 2020. Analysis of social media data using multimodal deep learning for disaster response. In *17th International Conference on Information Systems for Crisis Response and Management*. ISCRAM.

Seo Yeon Park and Cornelia Caragea. 2024. VerifyMatch: A semi-supervised learning paradigm for natural language inference with confidence-aware MixUp. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 19319–19335, Miami, Florida, USA. Association for Computational Linguistics.

Geoff Pleiss, Tianyi Zhang, Ethan R. Elenberg, and Kilian Q. Weinberger. 2020. Identifying mislabeled data using the area under the margin ranking. *Preprint*, arXiv:2001.10528.

Siyuan Qiao, Wei Shen, Zhishuai Zhang, Bo Wang, and Alan Yuille. 2018. Deep co-training for semi-supervised image recognition. *Preprint*, arXiv:1803.05984.

Antti Rasmus, Mathias Berglund, Mikko Honkala, Harri Valpola, and Tapani Raiko. 2015. Semi-supervised learning with ladder networks. *Advances in neural information processing systems*, 28.

Mobashir Sadat and Cornelia Caragea. 2024. Co-training for low resource scientific natural language inference. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2538–2550, Bangkok, Thailand. Association for Computational Linguistics.

Mehdi Sajjadi, Mehran Javanmardi, and Tolga Tasdizen. 2016. Regularization with stochastic transformations and perturbations for deep semi-supervised learning. *Advances in neural information processing systems*, 29:1163–1171.

Iustin Sirbu, Tiberiu Sosea, Cornelia Caragea, Doina Caragea, and Traian Rebedea. 2022. Multimodal semi-supervised learning for disaster tweet classification. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 2711–2723, Gyeongju, Republic of Korea. International Committee on Computational Linguistics.

Kihyuk Sohn, David Berthelot, Chun-Liang Li, Zizhao Zhang, Nicholas Carlini, Ekin D Cubuk, Alex Kurakin, Han Zhang, and Colin Raffel. 2020. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *arXiv preprint arXiv:2001.07685*.

Tiberiu Sosea and Cornelia Caragea. 2022. Leveraging training dynamics and self-training for text classification. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 4750–4762, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Tiberiu Sosea and Cornelia Caragea. 2023. Margin-match: Improving semi-supervised learning with pseudo-margins. *Preprint*, arXiv:2308.09037.

Antti Tarvainen and Harri Valpola. 2017. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. *Advances in neural information processing systems*, 30.

A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.

Yidong Wang, Hao Chen, Yue Fan, Wang Sun, Ran Tao, Wenxin Hou, Renjie Wang, Linyi Yang, Zhi Zhou, Lan-Zhe Guo, Heli Qi, Zhen Wu, Yu-Feng Li, Satoshi Nakamura, Wei Ye, Marios Savvides, Bhiksha Raj, Takahiro Shinozaki, Bernt Schiele, Jindong Wang, Xing Xie, and Yue Zhang. 2022. Usb: A unified semi-supervised learning benchmark for classification. *Preprint*, arXiv:2208.07204.

Yidong Wang, Hao Chen, Qiang Heng, Wenxin Hou, Yue Fan, Zhen Wu, Jindong Wang, Marios Savvides, Takahiro Shinozaki, Bhiksha Raj, Bernt Schiele, and Xing Xie. 2023. Freematch: Self-adaptive thresholding for semi-supervised learning. *Preprint*, arXiv:2205.07246.

Chen Wei, Kihyuk Sohn, Clayton Mellina, Alan Yuille, and Fan Yang. 2021. Crest: A class-rebalancing self-training framework for imbalanced semi-supervised learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10857–10866.

Qizhe Xie, Zihang Dai, Eduard Hovy, Minh-Thang Luong, and Quoc V. Le. 2020a. Unsupervised data augmentation for consistency training. *Preprint*, arXiv:1904.12848.

Qizhe Xie, Zihang Dai, Eduard Hovy, Thang Luong, and Quoc Le. 2020b. Unsupervised data augmentation for consistency training. *Advances in neural information processing systems*, 33:6256–6268.

Yi Xu, Lei Shang, Jinxing Ye, Qi Qian, Yu-Feng Li, Baigui Sun, Hao Li, and Rong Jin. 2021. Dash: Semi-supervised learning with dynamic thresholding. In *International conference on machine learning*, pages 11525–11536. PMLR.

Xingrui Yu, Bo Han, Jiangchao Yao, Gang Niu, Ivor Tsang, and Masashi Sugiyama. 2019. How does disagreement help generalization against label corruption? In *International conference on machine learning*, pages 7164–7173. PMLR.

Bowen Zhang, Yidong Wang, Wenxin Hou, Hao Wu, Jindong Wang, Manabu Okumura, and Takahiro Shinozaki. 2022. Flexmatch: Boosting semi-supervised learning with curriculum pseudo labeling. *Preprint*, arXiv:2110.08263.

H Zhang, M Cisse, Y Dauphin, and D Lopez-Paz. 2018. mixup: Beyond empirical risk management. In *6th Int. Conf. Learning Representations (ICLR)*, pages 1–13.

Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28.

Mingkai Zheng, Shan You, Lang Huang, Fei Wang, Chen Qian, and Chang Xu. 2022. Simmatch: Semi-supervised learning with similarity matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14471–14481.

Henry Peng Zou and Cornelia Caragea. 2023. Jointmatch: A unified approach for diverse and collaborative pseudo-labeling to semi-supervised text classification. *Preprint*, arXiv:2310.14583.

Henry Peng Zou, Yue Zhou, Weizhi Zhang, and Cornelia Caragea. 2023. DeCrisisMB: Debiased semi-supervised learning for crisis tweet classification via memory bank. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6104–6115, Singapore. Association for Computational Linguistics.

A Class-imbalanced Semi-supervised Learning

Existing semi-supervised learning algorithms usually assume a balanced class distribution of the data, which is not usually the case in real-world scenarios. Furthermore, while classifiers in general have a bias toward the majority classes, this poses a bigger problem for the SSL algorithms relying on pseudo-labeling. As the increasingly biased predictions are used for generating pseudo-labels, the bias increases even more. To address this, ABC (Lee et al., 2021) proposes an auxiliary balanced classifier that can be attached to an existing SSL algorithm in order to mitigate class imbalance. The auxiliary classifier is attached to the last features layer of the model so it can leverage useful representations learned by the backbone from the entire dataset. However, it is trained by masking the samples with a probability inversly proportional to the size of their corresponding class, thus enabling the use of a balanced classification loss. Note that the ABC loss is just a supplementary loss over the base SSL algorithm. This makes the approach different from plain undersampling of the majority class, as it is still used in it's entirety by the base SSL algorithm.

Due to the significant performance improvement shown by ABC on class-imbalanced datasets

from Computer Vision using FixMatch (Sohn et al., 2020) as a base SSL algorithm and due to its versatility, as it can be used in combination with any base SSL algorithm, we also include it in our experiments. As our MultiMatch approach obtains particularly impressive results over the baselines in highly imbalanced setups, we find it relevant to assess whether the gap between our approach and other SSL algorithms can be bridged simply by the integration of a balancing method such as ABC in the training pipeline.

To this end, for each SSL algorithm tested in the highly imbalanced setups, we perform additional experiments using the same algorithm in combination with ABC (Lee et al., 2021) and show the results in Table 2.

B MultiMatch for Class Imbalance

According to Gui et al. (2024), CISSL methods address the bias accumulation problem in SSL algorithms through two main strategies. The first approach, already described in Appendix A and exemplified by ABC (Lee et al., 2021), adjusts the classifier to mimic a balanced class distribution. In this setup, all unlabeled samples contribute to learning useful representations, but the classification loss is computed in a way that accounts for class imbalance – typically by weighting or masking examples based on class frequency. The second, more direct strategy aims to improve pseudo-label quality by reducing the influence of overrepresented classes. For instance, CReST (Wei et al., 2021) modifies the self-training paradigm with a re-sampling technique that prioritizes pseudo-labels from minority classes. Meanwhile, Adsh (Guo and Li, 2022) introduces an adaptive thresholding mechanism that enforces the use of a similar percentage of pseudo-labels for each class, mitigating imbalance amplification. Finally, reducing pseudo-label impurity – defined as the labeling error at each epoch – can help limit long-term error accumulation.

In this context, MultiMatch incorporates, by design, several CISSL techniques that explain its superior performance in highly imbalanced settings. First, the use of class-wise thresholds allows the algorithm to differentiate between the learning progress of individual classes. The impact of this thresholding strategy is evident in Table 2, by analyzing the performance of FreeMatch. While it performs similarly to FixMatch under balanced conditions, it shows a substantial improvement in imbalanced setups, reaching second place before applying the ABC enhancement.

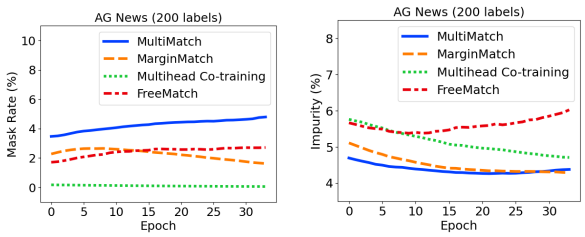


Figure 3: Mask rate (left) and impurity (right) on AG News with imbalance factor 100.

Second, MultiMatch uses a percentile-based threshold for each class, similar to Adsh, enabling a more equitable selection of pseudo-labels across classes. Although MarginMatch also employs a percentile-based threshold, it fails to achieve the same effect due to its lack of class-wise computation.

Finally, MultiMatch explicitly promotes lower pseudo-label impurity through several mechanisms. The use of multiple filters increases the difficulty for incorrect pseudo-labels to pass, while the subset employed for threshold computation results in generally higher thresholds compared to MarginMatch. As shown in Figure 3, MultiMatch maintains a low pseudo-label impurity – therefore mitigating error accumulation – using a higher mask rate of the predictions. While Sosea and Caragea (2023) point out that excessive masking can discard informative borderline examples, MultiMatch compensates for this by assigning a higher weight to the *useful & difficult* samples that pass all filtering criteria.

C Extended Background

FreeMatch (Wang et al., 2023) shows that using a fixed confidence threshold τ is suboptimal, as it does not account for varying difficulty across classes and for the model's increase in confidence over time. They address these issues by introducing class-wise thresholds, adaptively scaling $\tau(c)$ depending on the learning status of each class. Specifically, FreeMatch computes a self-adaptive global threshold τ_t that reflects the confidence of the model on the whole unlabeled dataset:

$$\tau_t = \lambda \tau_{t-1} + (1-\lambda)\frac{1}{\mu B}\sum_{b=1}^{\mu B}\max(q_b) \quad (7)$$

and a self-adaptive local threshold $\tilde{p}_t(c)$ that reflects the confidence of the model for a specific

class:

$$\tilde{p}_t(c) = \lambda \tilde{p}_{t-1}(c) + (1-\lambda)\frac{1}{\mu B}\sum_{b=1}^{\mu B} q_b(c) \quad (8)$$

with $\tau_0 = \tilde{p}_0(c) = \frac{1}{C}$.

Finally, the two thresholds are combined in order to obtain the final self-adaptive class thresholds $\tau_t(c)$ as the product between the normalized local thresholds and the global one.

$$\tau_t(c) = \frac{\tilde{p}_t(c)}{\max_{c'}(\tilde{p}_t(c'))} \cdot \tau_t \quad (9)$$

These thresholds are used to replace the filtering function from FixMatch with:

$$\mathbb{1}_{Free}^{(t)} = \mathbb{1}(\max(q_b) > \tau_t(\hat{q_b})) \quad (10)$$

D Experimental Setup

Shared Hyperparameter Settings. In order to ensure a straightforward and fair comparison with the algorithms already included in USB (Wang et al., 2022), we follow their experimental setup and use the same hyperparameters in all the experiments. More precisely, we use the pretrained uncased version of BERT-Base (Devlin, 2018) with a maximum text length of 512 tokens for all the algorithms. While suboptimal, we use no weak augmentation ($\alpha(x) = x$), to ensure backward compatibility. For strong augmentation we adopt back-translation (Xie et al., 2020b) with De-En and Ru-En translation using WMT19. While for the other 4 datasets the augmentations were already provided in USB so they were identical in all experiments, for IMDB we had to regenerate the translations, so there might be differences between the translations used by the 16 baselines from USB and the 3 methods implemented by us into the framework, due to the temperature 0.9 used for translation. The optimization with the AdamW optimizer with weight decay of 1e-4, cosine learning rate scheduler with a total training steps of 102, 400 and a warm-up of 5, 120 steps were preserved in all experiments. The optimal learning rates and layer decay rates for each dataset were determined by Wang et al. (2022) using a grid search for the FixMatch (Sohn et al., 2020) approach and were used unchanged for all approaches. The only change in setup compared to those reported in USB (Wang et al., 2022) was the use of batch size 8 instead of 4, as it was shown to improve performance [2]. The reported results for the baselines also used this improved batch size. Following USB, we also used the unlabeled loss weight $w_u = 1$ and the ration between unlabeled and labeled examples in a batch $\mu = 1$, although it was shown that a larger μ improves performance (Sohn et al., 2020; Sirbu et al., 2022).

MultiMatch Hyperparameters. The hyperparameters specific to MultiMatch are the number of heads H, which we set to 3 from the start, similar to Multihead Co-training, the weight w_d used for the *useful & difficult* samples, the percentile f used in computing our self-adaptive APM thresholds, and their lower bound γ_{min}. For the APM percentile f we experimented with values of 5% and 10%, and selected 5% as the optimal value. Setting f=10% determined an increased mask rate, which excluded relevant samples from the unsupervised loss computation. For the lower bound γ_{min}, we experimented with the values 0 and $-\infty$ (no lower bound), and found that $-\infty$ works better in highly imbalanced setups as it allows for more pseudo-labels from the underrepresented classes. For the weight w_d of the *useful & difficult* examples, we experimented with values of 3 and 5, selecting 3 as the optimal value. The hyperparameters selection was done following experiments on the AG News dataset with 200 labels and choosing the value that resulted in a smaller validation error averaged over 3 runs. The method was chosen due to the faster training time on AG News, but it does not exclude that different values might be optimal for other tasks. However, our selected values provided consistent performance across all setups.

Computational Resources. We conduct our experiments on a system with 8 Nvidia H100 GPUs with 80GB VRAM. Each experiment was run in a single GPU setting, the multiple GPUs being used to run multiple experiments simultaneously. Each training run took between 2 and 4 hours depending on the dataset (≈2h for AG News, ≈3h for IMDB and Yahoo, and ≈4h for Amazon and Yelp). With an average of ≈3h per run, 3 runs per setup, 10 setups (5 datasets, 2 sizes each) and 5 models (MultiMatch, MarginMatch, Multihead Co-training, SequenceMatch, CGMatch), the total training time for the main results table is about 3x3x10x5 = 450 GPU hours. For the imbalanced setups we trained 10 models (5 models with and without ABC) for a

[2] https://github.com/microsoft/Semi-supervised-learning/blob/main/results/usb_nlp.csv

Dataset	Label Type	#Classes	#Labeled	#Unlabeled	#Validation	#Test
IMDB	Movie Review Sentiment	2	10 / 50	11,500	1,000	12,500
AG News	News Topic	4	10 / 50	25,000	2,500	1,900
Amazon Review	Product Review Sentiment	5	50 / 200	50,000	5,000	13,000
Yahoo! Answer	QA Topic	10	50 / 200	50,000	5,000	6,000
Yelp Review	Restaurant Review Sentiment	5	50 / 200	50,000	5,000	10,000

Table 5: Dataset statistics and split information as number of examples per class.

total of 3x3x10x10 = 900 GPU hours). The ablation study required to train 5 additional models for a total of 3x3x10x5 = 450 GPU hours. Adding this up, the total cost was about 1,800 GPU hours, or 75 GPU days.

E Datasets

Statistics for the text classification datasets employed in this work are shown in Table 5.

For the highly imbalanced setups, we use between 1,000 and 10 labels per class according to a long-tail distribution, as described in Section 4. For the imbalance factor 100 we use 10 times more unlabeled data for each class compared to the labeled data. For the imbalance factor -100, the total number of unlabeled samples is still 10 times higher than the labeled one, but the long-tail distribution of the unlabeled set is reversed. The validation and test sets are the same as the balanced settings.

F Efficiency Considerations

While SSL algorithms may significantly boost the performance of a supervised model by leveraging additional unlabeled data, they typically lead to slower training, as more data must be processed. Specifically, for a supervised model, the total training time $\mathcal{T}$ is directly proportional to the number of epochs N_e and the number of labeled examples N_L (i.e., $\mathcal{T} \approx N_L \times N_e$). However, in the case of an SSL approach, since each epoch processes $(\mu + 1) \times N_L$ examples, the total training time becomes $\mathcal{T} \approx (\mu + 1) \times N_L \times N_e$. This applies to all SSL algorithms used in this work.

Furthermore, some design choices may introduce additional computational overhead. For instance, a traditional co-training method requires twice the time and memory, as it trains two models in parallel. While Multihead Co-Training addresses this by sharing a backbone across heads, the presence of three classification heads still implies extra computation through two additional output layers. However, classification heads are typically small, and the impact on runtime varies depending on the backbone. In our experiments using BERT as the shared backbone, the training overhead of Multihead Co-Training and MultiMatch remained consistently below 1% compared to FixMatch.

At inference time, models trained with SSL behave similarly to their supervised counterparts, as the same amount of data is passed through the model. However, the overhead of a multihead architecture still persists. In our setup, predictions are made by averaging the logits from all three heads, which leads to a minor inference overhead (approximately 1%) compared to single-head architectures. While this may be relevant for real-world deployments, the overhead can be removed after training by retaining only a single head during inference. This incurs a marginal performance drop of approximately 0.1% on our tasks, as the three heads converge to high agreement in the final training stages.

G Results

Due to space constraints, we removed the 7 weakest baselines from Table 1. The full version of Table 1, which contains all baselines, is provided in Table 6. A summary of the results, together with the citations for each SSL algorithm, is provided in Table 7. For the shared backbone, we used the uncased BERT (Devlin, 2018) in all experiments.

MultiMatch achieves statistically significant improvements over USB baselines, as confirmed by a Friedman test followed by the Nemenyi post-hoc test (Demšar, 2006), which is a standard methodology for comparing multiple classifiers across multiple datasets. The Friedman test first evaluates whether there are significant differences among the classifiers' rankings across datasets. Upon finding a statistically significant difference, the Nemenyi test is then applied to identify which pairs of classifiers differ significantly. Our results indicate that MultiMatch performs significantly better than USB baselines, highlighting the robustness and generalizability of our proposed approach across a diverse range of text classification tasks.

Dataset	IMDB		AG News		Amazon Review		Yahoo! Answer		Yelp Review		Mean	Fried.	Final
# Label	20	100	40	200	250	1000	500	2000	250	1000	error	rank	rank
Supervised (Full)	5.69±0.15	5.72±0.13	5.78±0.07	5.73±0.11	36.40±0.05	36.40±0.05	24.87±0.04	24.84±0.04	32.04±0.03	32.04±0.03	20.95	-	-
Supervised (Small)	20.31±2.79	14.02±1.22	15.06±1.08	14.25±0.97	52.31±1.28	47.53±0.69	37.43±0.29	33.26±0.10	51.22±0.98	46.71±0.37	33.21	-	-
Pseudo-Labeling	45.45±4.43	19.67±1.01	19.49±3.07	14.69±1.88	53.45±1.90	47.00±0.79	37.70±0.65	32.72±0.31	54.51±0.82	47.33±0.20	37.20	17.3	18
Mean Teacher	20.06±2.51	13.97±1.49	15.17±1.21	13.93±0.65	52.14±0.52	47.66±0.84	37.09±0.18	33.43±0.28	50.60±0.62	47.21±0.31	33.13	16.0	16
Π-Model	49.99±0.01	44.75±3.99	60.70±19.0	12.58±0.57	77.22±1.50	53.17±2.56	44.91±1.32	32.45±0.45	75.73±4.01	59.82±0.61	51.13	18.6	19
VAT	25.93±2.58	11.61±1.79	14.70±1.19	11.71±0.84	49.83±0.46	46.54±0.31	34.87±0.41	31.50±0.35	52.97±1.41	45.30±0.32	32.50	13.6	15
MixMatch	26.12±6.13	15.47±0.65	13.50±1.51	11.75±0.60	59.54±0.67	61.69±3.32	35.75±0.71	33.62±0.14	53.98±0.59	51.70±0.68	36.31	16.1	17
ReMixMatch	50.00±0.00	50.00±0.00	75.00±0.00	75.00±0.00	80.00±0.00	80.00±0.00	90.00±0.00	90.00±0.00	80.00±0.00	80.00±0.00	75.00	20.9	21
AdaMatch	8.09±0.99	7.11±0.20	11.73±0.17	11.22±0.95	46.72±0.72	42.27±0.25	32.75±0.35	30.44±0.31	45.40±0.96	40.16±0.49	27.59	6.5	5
UDA	49.97±0.04	50.00±0.00	41.00±24.9	53.68±30.1	60.76±13.6	68.38±16.4	71.30±26.4	70.50±27.5	69.33±15.0	66.95±18.4	60.19	19.6	20
FixMatch	7.72±0.33	7.33±0.13	30.17±1.87	11.71±1.95	47.61±0.83	43.05±0.54	33.03±0.49	30.51±0.53	46.52±0.94	40.65±0.46	29.83	10.2	11
FlexMatch	7.82±0.77	7.41±0.38	16.38±3.94	12.08±0.73	45.73±1.60	42.25±0.33	35.61±1.08	31.13±0.18	43.35±0.69	40.51±0.34	28.23	9.4	9
Dash	8.34±0.86	7.55±0.35	31.67±13.1	13.76±1.67	47.10±0.74	43.09±0.60	35.26±0.33	31.19±0.29	45.24±2.02	40.14±0.79	30.33	11.7	13
CRMatch	8.96±0.88	7.16±0.09	12.28±1.43	11.08±1.24	45.49±0.98	43.07±0.50	32.51±0.40	29.98±0.07	45.71±0.63	40.62±0.28	27.69	7.6	6
CoMatch	7.44±0.30	7.72±1.14	11.95±0.76	10.75±0.35	48.76±0.90	43.36±0.21	33.48±0.51	30.25±0.35	45.40±1.12	40.27±0.51	27.94	8.1	8
SimMatch	7.93±0.55	7.08±0.33	14.26±1.51	12.45±1.37	45.91±0.95	42.21±0.30	33.06±0.20	30.16±0.21	46.12±0.48	40.26±0.62	27.94	7.6	6
FreeMatch	8.94±0.21	7.95±0.45	12.98±0.58	11.73±0.63	46.41±0.60	42.64±0.06	32.77±0.26	30.32±0.18	47.95±1.45	40.37±1.00	28.21	9.7	10
SoftMatch	7.76±0.58	7.97±0.72	11.90±0.27	11.72±1.58	45.29±0.95	42.21±0.20	33.07±0.31	30.44±0.62	44.09±0.50	39.76±0.13	27.42	6.4	4
Multihead Co-training	8.70±0.88	7.46±0.68	22.72±6.09	13.48±1.51	46.22±0.70	43.07±0.79	35.17±1.58	30.81±0.28	46.46±1.28	40.79±0.49	29.49	12.2	14
MarginMatch	7.19±0.39	6.99±0.19	**10.65±0.19**	11.03±0.99	44.81±1.23	42.14±0.67	32.08±0.70	29.55±0.15	42.93±1.48	39.13±0.34	26.65	2.3	2
SequenceMatch	12.99±7.64	8.34±0.57	16.17±2.53	12.56±0.87	47.97±2.39	42.58±0.21	34.92±0.78	30.28±0.28	45.60±0.84	40.44±0.51	29.19	11.3	12
CGMatch	7.07±0.36	**6.79±0.10**	11.95±0.63	11.29±0.47	44.77±1.39	42.61±0.47	32.15±0.58	29.85±0.16	44.34±0.26	40.14±0.07	27.10	3.9	3
MultiMatch	**6.89±0.07**	6.98±0.27	11.14±0.96	**10.59±0.66**	**44.43±0.98**	**42.09±0.28**	**30.90±0.70**	**29.39±0.39**	**42.16±0.79**	**39.08±0.55**	**26.37**	**1.2**	**1**

Table 6: Test error rates on IMDB, AG News, Amazon Review, Yahoo! Answer, and Yelp Review datasets using two setups with different sizes for the labeled set. The best result for each setup is highlighted in **blue**. The upper section of the table contains results reported by the USB benchmark, while the bottom section contains baselines integrated by us into the USB codebase.

Algorithm name	Citation	Mean error	Friedman rank	Final rank
BERT - Supervised (Full)	Devlin (2018)	20.95	-	-
BERT - Supervised (Small)	Devlin (2018)	33.21	-	-
Pseudo-Labeling	Lee et al. (2013)	37.20	17.3	18
Mean Teacher	Tarvainen and Valpola (2017)	33.13	16.0	16
Π-Model	Rasmus et al. (2015)	51.13	18.6	19
VAT	Miyato et al. (2018)	32.50	13.6	15
MixMatch	Berthelot et al. (2019b)	36.31	16.1	17
ReMixMatch	Berthelot et al. (2019a)	75.00	20.9	21
AdaMatch	Berthelot et al. (2021)	27.59	6.5	5
UDA	Xie et al. (2020a)	60.19	19.6	20
FixMatch	Sohn et al. (2020)	29.83	10.2	11
FlexMatch	Zhang et al. (2022)	28.23	9.4	9
Dash	Xu et al. (2021)	30.33	11.7	13
CRMatch	Fan et al. (2023)	27.69	7.6	6
CoMatch	Li et al. (2021)	27.94	8.1	8
SimMatch	Zheng et al. (2022)	27.94	7.6	6
FreeMatch	Wang et al. (2023)	28.21	9.7	10
SoftMatch	Chen et al. (2023)	27.42	6.4	4
Multihead Co-training	Chen et al. (2021)	29.49	12.2	14
MarginMatch	Sosea and Caragea (2023)	26.65	2.3	2
SequenceMatch	Nguyen (2024)	29.19	11.3	12
CGMatch	Cheng et al. (2025)	27.10	3.9	3
MultiMatch	Ours	**26.37**	**1.2**	**1**

Table 7: Citations for all the methods used in our experiments as strong baselines.

H Comparison to Large Language Models

Although a direct comparison between specialized SSL methods and general-purpose LLMs is inherently challenging (e.g., due to potential data contamination and significant disparity in model sizes), we include this analysis to provide a broader context for our proposed approach. These experiments are particularly valuable for assessing the performance of LLMs in specialized classification tasks and for highlighting the continued relevance of specialized, inference-efficient models in a landscape dominated by LLMs. To this end, we evaluate a representative LLM, GPT-4o-mini, on the five NLP datasets used in this work and show the results in Table 8.

Dataset	Test Error (%)
IMDB	5.34
AG News	16.26
Amazon Review	36.42
Yahoo! Answer	30.96
Yelp Review	32.96

Table 8: Performance of GPT-4o-mini on the USB datasets.

For the three sentiment classification tasks (IMDB, Amazon Review, and Yelp Review), the performance of GPT-4o-mini is comparable to the Supervised (Full) baseline presented in Table 1, which was trained with large quantities of labeled data (e.g. 250k samples for Amazon and Yelp). This strongly supports the hypothesis that such widely available datasets have been included in LLM training data, making them unsuitable for a fair comparison between SSL and LLMs.

Notably, MultiMatch significantly outperforms GPT-4o-mini on the AG News dataset. To understand the reasons behind this, we perform a quantitative and qualitative analysis of the LLM's predictions. The confusion matrix shown in Table 9 reveals that the model frequently misclassifies news from the *Sci/Tech* category as *Business*, and occasionally *World* news as *Business*. A qualitative analysis shows that a large portion of the *Sci/Tech* news involves corporate activities (e.g., acquisitions, mergers, layoffs, new project announcements). This semantic overlap with the *Business* category leads to misclassification by the general-purpose LLM. Similarly, some *World* news regarding inflation rates or trade agreements between governments are also mistakenly classified as *Business*. These findings highlight the need for specialized training to distinguish between such nuanced categories, where SSL methods like MultiMatch prove effective even under limited labeled data.

True \ Pred	World	Sports	Business	Sci/Tech
World	0.87	0.03	0.09	0.02
Sports	0.01	0.98	0.01	0.00
Business	0.03	0.00	0.92	0.04
Sci/Tech	0.07	0.01	0.34	0.58

Table 9: Confusion matrix of GPT-4o-mini predictions on AG News.

While these experiments provide valuable insights, we re-emphasize that a direct comparison between our SSL framework and a general-purpose LLM is inherently flawed. First, such a comparison needs to be performed in-depth, using novel datasets not seen during LLM pretraining. Secondly, our framework demonstrates the effectiveness of SSL for training small specialized models using limited labeled data. Such models have a significantly lower inference complexity than LLMs of billions of parameters, making them highly suitable for resource-constrained systems. As detailed in Appendix F, a model trained with SSL preserves the inference complexity of the underlying backbone (e.g., BERT). Therefore, the SSL domain remains highly relevant, providing a complementary approach to LLMs by offering high-performance solutions for specialized, resource-limited tasks.

I Code and License

The source code for our MultiMatch method and all experimental configurations are publicly available on GitHub at https://github.com/iustinsirbu13/MultiMatch. Our implementation is a public fork of the Unified Semi-supervised Learning (USB) framework (Wang et al., 2022) and is released under the MIT license, consistent with the original framework. We have submitted a pull request to the official USB repository to integrate MultiMatch as a new algorithm, thereby contributing our work to the open-source community.